



Міністерство освіти і науки України
Національний технічний університет України
“Київський політехнічний інститут імені Ігоря Сікорського”
Факультет інформатики та обчислювальної техніки
Кафедра інформаційних систем та технологій

Лабораторна робота №7
з дисципліни «Технології розроблення
програмного забезпечення»
Тема: «Патерни проектування»
Варіант «Mind-mapping software»

Виконав:

студент групи ІА-32

Бурдін Д. Є.

Перевірив:

Мягкий М. Ю.

Тема: Патерни проектування.

Мета: Вивчити структуру шаблонів «Mediator», «Facade», «Bridge», «Template method» та навчитися застосовувати їх в реалізації програмної системи.

Тема проєкту: Mind-mapping software

Патерни: Strategy, Prototype, Abstract Factory, Bridge, Composite, SOA

Опис: Візуальний додаток для складання "карт пам'яті" з можливістю роботи з декількома картами (у вкладках), автоматичного промальовування ліній, додавання вкладених файлів, картинок, відеофайлів (попередній перегляд); можливість додавання значків категорій / терміновості, обведення областей карти (поділ пунктирною лінією)

Зміст

Теоретичні відомості.....	3
Хід роботи	4
Діаграма класів патерну Bridge	5
Код програми	5
Висновок	11
Питання до лабораторної роботи	11

Теоретичні відомості

Шаблон «Bridge»

Призначення патерну: Шаблон «Bridge» (міст) використовується для поділу інтерфейсу і його реалізації. Це необхідно у випадках, коли може існувати кілька різних абстракцій, над якими можна проводити дії різними способами.

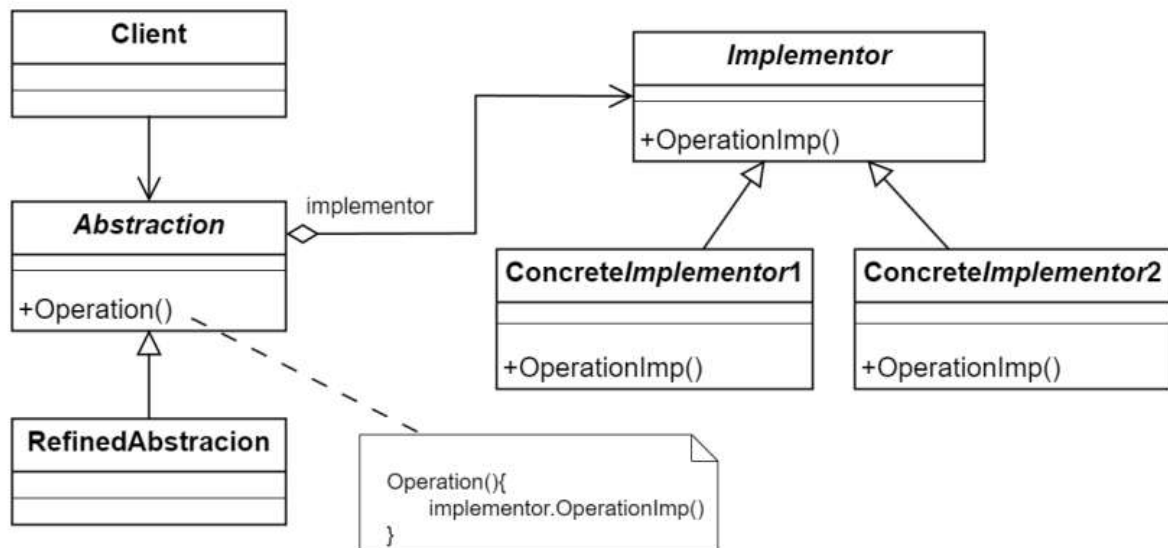


Рис. 1. Структура патерну «Міст»

Проблема: Ви реалізовуєте графічний векторний редактор, який дозволяє рисувати кола, прямокутники, прямі та довільні лінії.

Ви маєте реалізувати функціонал відображення отриманого рисунку на екрані та друкувати на принтер.

Спростимо ситуацію: ваш редактор дозволяє рисувати лише лінії та кола.

При такому підході, нам потрібно будувати ієрархію фігур з різною реалізацією дочірніх класів: `LinePrint`, `LineDraw`, `CirclePrint`, `CircleDraw`.

Якщо додати прямі, то добавиться ще два підкласи, і т.д. А як бути, коли нам потрібно буде ще і реалізувати збереження в `bitmap` форматі? добавляємо ще `LineBinary`, `CircleBinary`? При такому підході ми отримуємо дуже складну ієрархію класів.

Рішення: В даному випадку ми можемо використати патерн «Міст» (Bridge): робимо дві ієрархії – фігур (`Shape`) та рисування (`DrawApi`).

При такому підході `DrawApi` – це інтерфейс імплементції відображення (графічного драйвера), а `Shape` – інтерфейс абстракції фігур, яка має

агрегацію з об'єктом DrawApi. При такому підході фігури будуть делегувати рисуння об'єкту DrawApi.

Лінія, коло, та інші будуть дочірніми класами до Shape, а WindowDrawApi та PrinterDrawApi – дочірні класи до DrawApi, які представляють графічні драйвери для відображення на екрані та принтері відповідно. Якщо нам потрібно буде додати ще і збереження в bitmap форматі, то ми додаммо ще один підклас реалізації графічного драйвера BitmapDrawApi. Таким чином ми маємо дві різні ієрархії об'єктів і вони в нас не перетинаються і не збільшуються в геометричній прогресії при додаванні нових драйверів або фігур.

Також слід відмітити, що DrawApi нічого не знає про фігури (абстракцію), а дочірні класи абстракції не залежать від реалізації графічного драйвера.

Переваги та недоліки:

- + Дозволяє змінювати ієрархії абстракції та реалізації незалежно одна від одної.
- + Розділивши абстракцію від реалізації отримуємо більшу гнучкість та простіший супровід такого коду.
- Підвищена гнучкість при використанні патерну отримується за рахунок більшої складності, введення додаткових проміжних рівнів.

Хід роботи

Завдання

- Ознайомитись з короткими теоретичними відомостями.
- Реалізувати частину функціоналу робочої програми у вигляді класів та їхньої взаємодії для досягнення конкретних функціональних можливостей.
- Реалізувати один з розглянутих шаблонів за обраною темою.
- Реалізувати не менше 3-х класів відповідно до обраної теми.
- Підготувати звіт щодо виконання лабораторної роботи. Поданий звіт повинен містити: діаграму класів, яка представляє використання шаблону в реалізації системи, навести фрагменти коду по реалізації цього шаблону.

Діаграма класів патерну Bridge

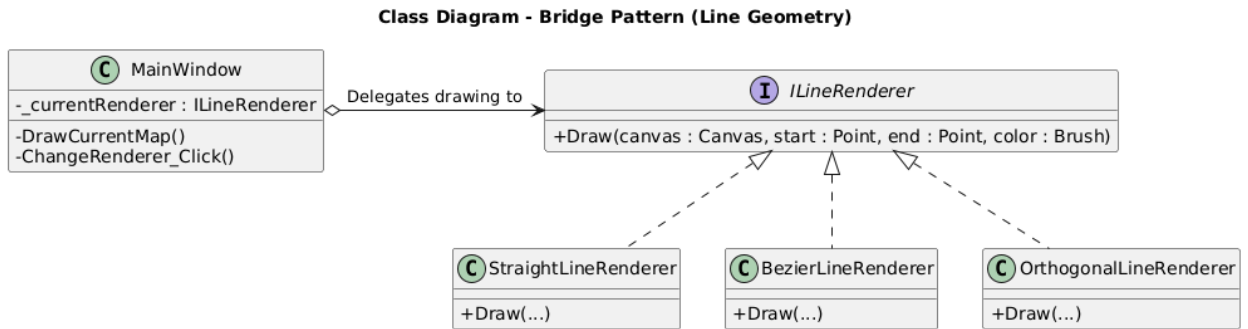


Рис. 2. Діаграма класів патерну Bridge

Код програми

ILineRenderer

```
using System.Collections.Generic;
using System.Windows;
using System.Windows.Controls;
using System.Windows.Media;
using MindMapApp.Entities;
```

```
namespace MindMapApp.Renderers
```

```
{
```

```
    public interface ILineRenderer
```

```
    {
```

```
        void Draw(Canvas canvas, Point start, Point end, Brush color);
```

```
    }
```

```
}
```

StraightLineRenderer

```
using MindMapApp.Entities;
using System.Windows;
using System.Windows.Controls;
using System.Windows.Media;
using System.Windows.Shapes;
```

```

namespace MindMapApp.Renderers
{
    public class StraightLineRenderer : ILineRenderer
    {
        public void Draw(Canvas canvas, Point start, Point end, Brush color)
        {
            var line = new Line
            {
                X1 = start.X,
                Y1 = start.Y,
                X2 = end.X,
                Y2 = end.Y,
                Stroke = color,
                StrokeThickness = 2
            };
            Panel.SetZIndex(line, -1);
            canvas.Children.Add(line);
        }
    }
}

```

BezierLineRenderer

```

using MindMapApp.Entities;
using System;
using System.Windows;
using System.Windows.Controls;
using System.Windows.Media;
using System.Windows.Shapes;

```

```

namespace MindMapApp.Renderers
{
    public class BezierLineRenderer : ILineRenderer
    {
        public void Draw(Canvas canvas, Point start, Point end, Brush color)
        {
            double deltaX = Math.Abs(end.X - start.X);
            double deltaY = Math.Abs(end.Y - start.Y);

            Point p1, p2;

            if (deltaX > deltaY)
            {
                double offset = deltaX / 2;
                p1 = new Point(start.X + offset, start.Y);
                p2 = new Point(end.X - offset, end.Y);
            }
            else
            {
                double offset = deltaY / 2;
                p1 = new Point(start.X, start.Y + offset);
                p2 = new Point(end.X, end.Y - offset);
            }

            var bezierSegment = new BezierSegment(p1, p2, end, true);
            var pathFigure = new PathFigure(start, new[] { bezierSegment }, false);
            var geometry = new PathGeometry(new[] { pathFigure });

            var path = new Path

```

```

    {
        Data = geometry,
        Stroke = color,
        StrokeThickness = 2
    };

    Panel.SetZIndex(path, -1);
    canvas.Children.Add(path);
}
}
}

```

OrthogonalLineRenderer

```

using System;
using System.Collections.Generic;
using System.Windows;
using System.Windows.Controls;
using System.Windows.Media;
using System.Windows.Shapes;
using MindMapApp.Entities;

namespace MindMapApp.Renderers
{
    public class OrthogonalLineRenderer : ILineRenderer
    {
        public void Draw(Canvas canvas, Point start, Point end, Brush color)
        {
            var points = new PointCollection();
            points.Add(start);
            Point C = new Point(start.X, end.Y);

```



```

double dx_SE = end.X - start.X;
double dy_SE = end.Y - start.Y;
double dx_SC = C.X - start.X;
double dy_SC = C.Y - start.Y;
double denominator = (dx_SE * dx_SE) + (dy_SE * dy_SE);
double intersectionY;

if (denominator < 0.0001)
{
    intersectionY = start.Y;
}
else
{
    double t = (dx_SC * dx_SE) + (dy_SC * dy_SE) / denominator;
    intersectionY = start.Y + t * dy_SE;
}
points.Add(new Point(start.X, intersectionY));
points.Add(new Point(end.X, intersectionY));
points.Add(end);
var polyline = new Polyline
{
    Stroke = color,
    StrokeThickness = 2,
    Points = points,
    StrokeLineJoin = PenLineJoin.Round
};

Panel.SetZIndex(polyline, -10);
canvas.Children.Add(polyline);

```

}

}

}

Висновок: Під час виконання лабораторної роботи було реалізовано патерн «Міст» це дозволило, відокремити абстракції від їх реалізації, що дозволило змінювати вигляд і метод рендерингу ліній зв'язку динамічно під час роботи програми

Питання до лабораторної роботи

1. Яке призначення шаблону «Посередник»?
Зменшити зв'язність між класами, виносячи їхню взаємодію в окремий об'єкт-посередник. Це дозволить об'єктам не посилатися один на одного напряму, а спілкуватися лише через посередника, що спрощує підтримку коду та повторне використання компонентів.
2. Які класи входять в шаблон «Посередник», та яка між ними взаємодія? Посередник, Конкретний посередник та класи Колег (компонентів); взаємодія відбувається так, що Колеги повідомляють про події Посередника, а той вирішує, кому перенаправити запит або яку дію виконати, виключаючи пряме спілкування між компонентами.
3. Яке призначення шаблону «Фасад»?
Надати простий та уніфікований інтерфейс до складної системи класів, бібліотеки або фреймворку, що дозволяє приховати від клієнта деталі внутрішньої реалізації та складні залежності підсистеми
4. Які класи входять в шаблон «Фасад», та яка між ними взаємодія? Фасад та класи Підсистеми. Взаємодія полягає в тому, що клієнт звертається до методів Фасаду, який знає, яким класам переадресувати запит, і делегує їм виконання роботи, при цьому самі класи підсистеми нічого не знають про існування Фасаду.
5. Яке призначення шаблону «Міст»?
Розділення абстракції та її реалізації таким чином, щоб вони могли змінюватися та розширюватися незалежно одна від одної, що дозволяє уникнути комбінаторного вибуху кількості класів при розвитку системи у кількох вимірах.
6. Які класи входять в шаблон «Міст», та яка між ними взаємодія? Абстракція, Уточнена абстракція, інтерфейс Реалізації та Конкретні реалізації. взаємодія будується на тому, що Абстракція містить посилання на об'єкт Реалізації та делегує йому виконання низькорівневих операцій, не знаючи деталей виконання.
7. Яке призначення шаблону «Шаблонний метод»?
Визначення скелета алгоритму в суперкласі із делегуванням реалізації окремих його кроків підкласам, що дозволяє змінювати частини

алгоритму без порушення його загальної структури та послідовності виконання.

8. Які класи входять в шаблон «Шаблонний метод», та яка між ними взаємодія?

Абстрактний клас, що містить шаблонний метод і декларацію кроків, та Конкретні класи, що реалізують ці кроки; взаємодія відбувається через виклик шаблонного методу, який послідовно запускає кроки, специфічна реалізація яких міститься у класах-спадкоємцях.

9. Чим відрізняється шаблон «Шаблонний метод» від «Фабричного методу»?

Шаблонний метод є поведінковим патерном і керує структурою виконання алгоритму в цілому, а Фабричний метод є породжуючим патерном і спеціалізується лише на делегуванні процесу створення об'єкта підкласам.

10. Яку функціональність додає шаблон «Міст»?

Динамічна зміна реалізації під час виконання програми, оскільки зв'язок між абстракцією та реалізацією базується на композиції (посиланні), а не на жорсткому успадкуванні.